

TASK 02 — API Rate Limiting & Throttling

Step 1 — Study rate limiting

Rate Limiting

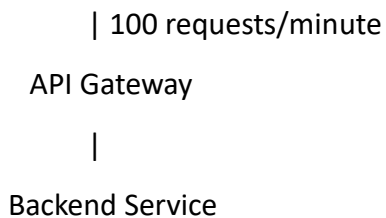
Rate limiting controls the **number of requests** allowed during a particular time period.

Example: 100 requests / minute / user

It protects the API from:

- Excessive traffic
- Server overload
- Automated scripts
- API abuse
- Denial-of-service-like traffic

Example User A



If User A sends 150 requests:

First 100 → Allowed

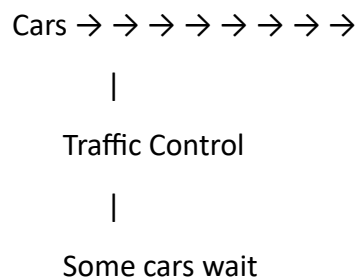
Next 50 → Blocked/Delayed

Throttling

Throttling means slowing down or restricting requests when traffic becomes too high.

Think of it like a traffic signal.

If too many cars are trying to enter a road:



Similarly:

Clients

|

API Gateway

|

Rate Limiter

|— Allowed → Backend

└— Too fast → Delay/Reject

Difference

Rate Limiting

Sets a request limit

Example: 100/minute

Usually rejects requests after limit

Protects resources

Throttling

Controls request speed

Example: slow excessive traffic

May delay, queue, or reject

Controls traffic flow

Request Quota

A **request quota** defines how many requests a client is allowed to make during a longer period.

Example:

10,000 requests / day / user

Suppose a user has:

Daily quota = 10,000

Used = 9,950

Remaining = 50

The user can make only **50 more requests** before reaching the quota.

Rate limit vs quota

Rate limit: 100 requests/minute

Quota: 10,000 requests/day

A user could make 100 requests every minute, but once the daily quota is exhausted, further requests are blocked until the quota resets.

Burst Traffic

Burst traffic means a sudden large number of requests arrive within a short period.

Example:

Normally:

10 requests/second

Suddenly:

1,000 requests/second

This is a traffic burst.

Example

Imagine an e-commerce website.

At 10:00 AM:

Normal traffic

↓

100 requests/sec

At 10:01 AM, a popular product becomes available:

Thousands of users

↓

API Gateway

↓

Huge traffic burst

Without protection, the backend could become overloaded.

Rate limiting and throttling help control this.

API Abuse

API abuse happens when someone intentionally or unintentionally uses an API excessively or improperly.

Examples:

Brute-force login attempts

POST /login

POST /login

POST /login

POST /login

...

An attacker may try thousands of passwords.

Rate limiting can reduce this.

Example:

5 login attempts / minute / IP

Product search abuse

Someone could continuously call:

GET /products?search=laptop

thousands of times.

This consumes server resources.

Rate limiting can control it.

Step 2 — Identify APIs That Require Protection

Login API

Example:

POST /api/login

This should have **strict rate limiting** because attackers may perform brute-force attacks.

Example:

5 requests / minute / IP

or:

10 requests / minute / user

Order Creation API

Example:

POST /api/orders

This API creates an order, so it should be protected from repeated requests.

Example:

20 requests / minute / user

It is also important to use **idempotency** so that accidentally repeated requests don't create duplicate orders.

Payment API

Example:

POST /api/payment

This is a critical API.

It should have strong protection because payment operations are sensitive.

Example:

10 requests / minute / user

Additional controls can include:

- Authentication
- Authorization
- Idempotency keys
- Fraud detection
- Logging
- Monitoring

Product Search API

Example:

GET /api/products?search=phone

Search APIs usually receive many requests.

Therefore, the limit can be higher.

Example:

100 requests / minute / user

Step 3 — Decide a Rate-Limit Rule

100 requests / minute / user

Let's understand it.

100

Maximum number of requests:

100

minute

Time window:

1 minute

user

The limit is applied separately to each user.

For example:

User A → 100 requests/minute

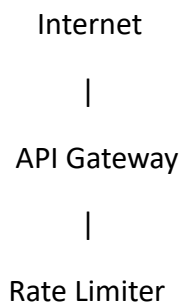
User B → 100 requests/minute

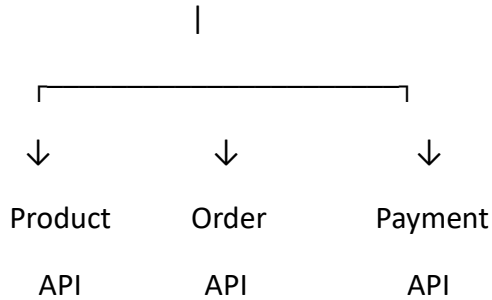
User C → 100 requests/minute

User A reaching the limit does **not necessarily block User B**.

Complete Example

Suppose we have an e-commerce application:





We could define:

API	Rate Limit
Login	5/minute/IP
Product Search	100/minute/user
Order Creation	20/minute/user
Payment	10/minute/user

This means each API gets a different protection level.

What Happens When the Limit Is Exceeded

Suppose:

Product API

100 requests/minute/user

User sends:

Requests 1–100 → Allowed

Request 101 → Rejected

The API can respond:

HTTP 429 Too Many Requests

Example response:

```
{
  "error": "Too many requests",
  "message": "Rate limit exceeded",
```

```
"retryAfter": 30
}
```

The client should wait and try again later.

Important Rate-Limiting Algorithms

You will commonly see these algorithms:

1. Fixed Window

Example:

10:00:00 → 10:01:00

Allow:

100 requests

Then the counter resets.

2. Sliding Window

Instead of using fixed time blocks, it continuously looks at the recent time period.

Example:

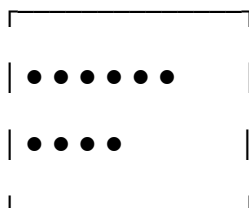
Last 60 seconds

This provides smoother rate limiting.

3. Token Bucket

A bucket contains tokens.

Token Bucket



|

Request



Token used

Each request consumes a token.

Tokens are added at a fixed rate.

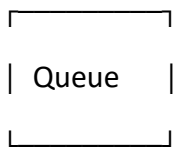
This algorithm can allow controlled **bursts** while still enforcing an average rate.

4. Leaky Bucket

Requests enter a queue and are processed at a controlled rate.

Requests

↓ ↓ ↓ ↓ ↓



Controlled

processing

Backend

This is useful when you want smoother traffic.

Why Rate Limiting Is Important

Rate limiting provides:

1. Protection

Prevents backend services from receiving unlimited requests.

2. Security

Helps reduce:

- Brute-force attacks
- API abuse
- Automated attacks
- Excessive scraping

3. Availability

Keeps the application available for legitimate users.

4. Cost Control

For cloud applications, excessive API calls can increase infrastructure costs.

5. Fair Usage

One user cannot easily consume all available resources.

Simple Real-World Example

Imagine a college API:

GET /students

Rule:

100 requests/minute/user

User makes:

09:00:01 → Request 1

09:00:10 → Request 20

09:00:20 → Request 50

09:00:40 → Request 100

09:00:45 → Request 101

The 101st request can receive:

429 Too Many Requests

After the rate-limit window resets, requests can be accepted again.

Step 4 — Configure Redis for Rate-Limit Storage

What is Redis

Redis is a very fast, in-memory data store.

For rate limiting, Redis is useful because the API Gateway needs to frequently:

- Check how many requests a user has made
- Increase the request counter

- Store expiration time
- Check whether the limit has been reached

Example rule:

100 requests / minute / user

Redis can store something like:

Key: rate:user:101

Value: 75

TTL: 35 seconds

Meaning:

User ID = 101

Requests = 75

Time remaining = 35 seconds

Why Redis

Imagine 1,000 users are accessing your application.

The Gateway needs to continuously check:

User A → 45 requests

User B → 78 requests

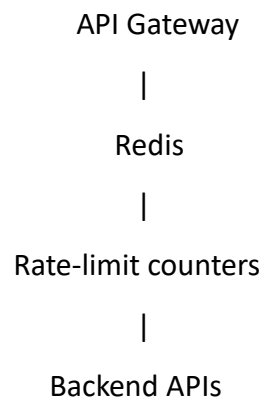
User C → 12 requests

User D → 100 requests

...

Redis is designed for this type of very fast counter operation.

Simple architecture



Redis is generally used as a **shared central store**, so multiple gateway instances can use the same rate-limit information.

Example Redis Data

Suppose the rule is:

100 requests / minute / user

User 101 makes 20 requests.

Redis might contain:

rate:user:101 → 20

User 102 makes 50 requests:

rate:user:102 → 50

User 103 makes 100 requests:

rate:user:103 → 100

When the time window expires, the counter can expire automatically.

Step 5 — Implement Rate Limiting at the API Gateway

Request

A client sends an API request.

For example:

GET /api/products

The request first reaches the **API Gateway**.

Client

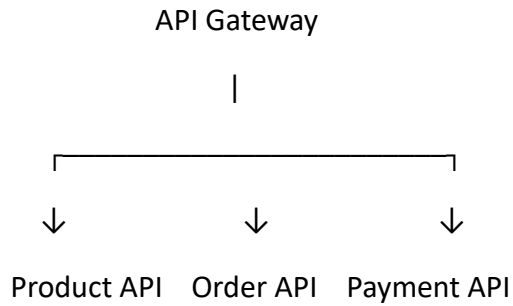
| GET /api/products

API Gateway

API Gateway

The API Gateway acts as the entry point to your microservices.

For example:



Before forwarding the request, the Gateway checks the rate limit.

Check Redis

The Gateway asks Redis:

"Has this user exceeded the allowed number of requests?"

For example:

Limit = 100 requests/minute

Redis currently says:

User 101

Current requests = 75

Therefore:

$75 < 100$

The request is allowed.

Limit Available?

The Gateway makes a decision.

YES

If the user still has requests available:

Limit Available?

|

YES

|

Forward Request

Example:

Current = 75

Limit = 100

$75 < 100$

Therefore:

Request → Allowed

The Gateway forwards the request to the appropriate microservice.

Client



API Gateway



Redis



Product Service



Response

5. Limit Not Available

Suppose the user has already used the complete limit:

Current = 100

Limit = 100

The next request arrives:

Client



API Gateway



Redis



Limit Available?

↓

NO

The Gateway should **not forward the request** to the backend.

Instead, it returns:

429 Too Many Requests

Step 6 — Return HTTP 429

HTTP status code **429** means:

The client has sent too many requests in a given amount of time.

Example response:

HTTP/1.1 429 Too Many Requests

Content-Type: application/json

Retry-After: 30

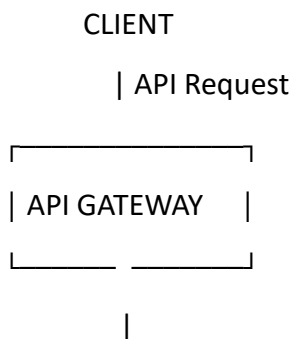
Response body:

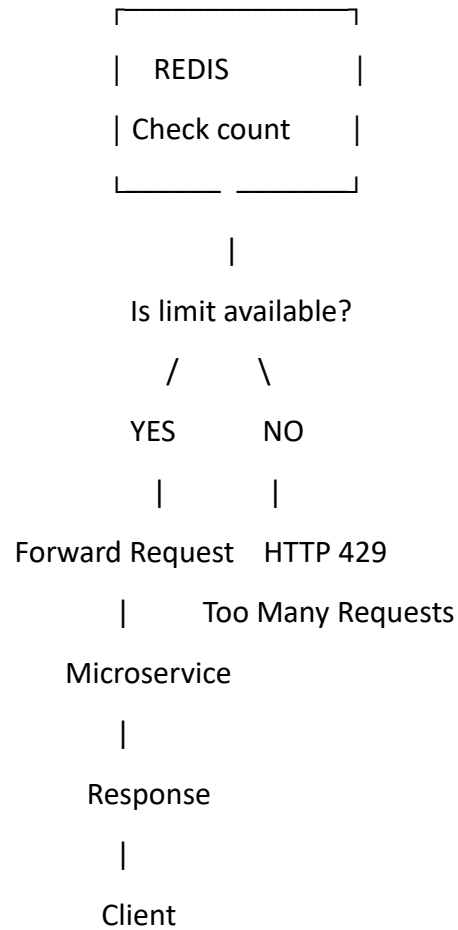
```
{  
  "error": "Too Many Requests",  
  "message": "Rate limit exceeded",  
  "retryAfter": 30  
}
```

The Retry-After value tells the client approximately how long it should wait before trying again.

Complete Flow

The entire process can be understood like this:





Example with 100 Requests/Minute

Suppose: Rate limit = 100 requests/minute/user

Requests 1–99

Request → Gateway → Redis → Allowed → Backend

Request 100

Request → Gateway → Redis → Allowed → Backend

Request 101

Request → Gateway → Redis

↓

Limit reached

↓

HTTP 429

The backend does **not** need to process request 101.

Example Redis Counter

A simplified implementation could use a Redis key such as:

rate:user:101

Suppose:

rate:user:101 = 98

New request:

$98 + 1 = 99$

Allowed.

Next request:

$99 + 1 = 100$

Allowed.

Next request:

$100 + 1 = 101$

Limit exceeded → reject with **429**.

Why Put Rate Limiting at the Gateway

This is especially useful in a microservices architecture.

Without gateway rate limiting:

Client

└─ Product Service

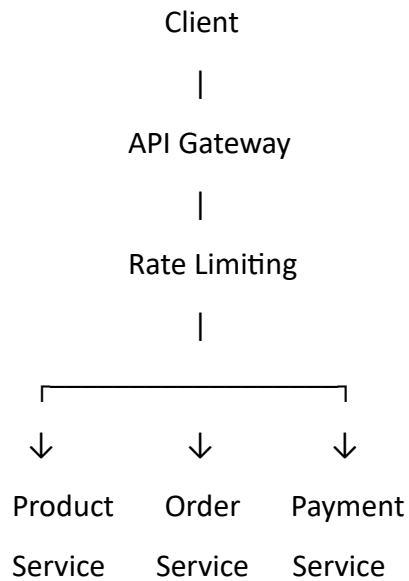
└─ Order Service

└─ Payment Service

└─ User Service

Every service would need its own rate-limiting logic.

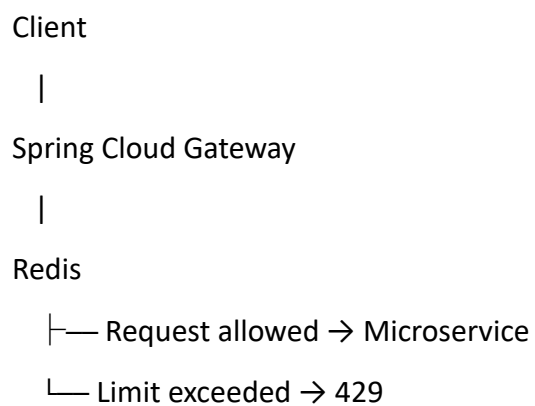
With a Gateway:



The Gateway provides a **central point of protection**.

Simple Spring Boot Concept

If you're implementing this in your **Spring Boot microservices project**, the architecture can look like:



A typical technology stack is:

Spring Boot
+
Spring Cloud Gateway
+
Redis
+
Microservices

Important Terms to Remember

Term	Meaning
Redis	Fast in-memory store used for counters/state
Rate Limit	Maximum requests allowed in a period
API Gateway	Entry point for API requests
Counter	Number of requests made
TTL	Time until a Redis key expires
HTTP 429	Too many requests
Retry-After	Tells client when to retry
Throttling	Controls excessive request traffic

Step 7 — Test Normal Traffic

We need to verify that **normal requests that are within the configured limit are successful.**

Suppose you configured:

100 requests / minute / user

Now send a few requests from Postman.

Example:

GET http://localhost:8080/api/products

You might send:

Request 1 → 200 OK

Request 2 → 200 OK

Request 3 → 200 OK

Request 4 → 200 OK

Request 5 → 200 OK

Because these requests are within the limit, they should be allowed.

Flow

Postman

|

API Gateway

|

Redis

|

Limit available?

|

YES

|

Product Service

|

200 OK

What should you check

Verify that:

1. The request reaches the Gateway.
2. Redis checks the request count.
3. The request is below the limit.
4. Gateway forwards the request.
5. The backend service processes it.
6. Postman receives a successful response.

Example evidence

Postman:

GET /api/products

Status: 200 OK

This proves that **normal traffic works**.

Step 8 — Test Excessive Requests

Now We need to test what happens when a user sends **too many requests**.

Suppose your configuration is:

10 requests / minute / user

Now send repeated requests using Postman.

For example:

Request 1

Request 2

Request 3

...

Request 10

Request 11

Request 12

The first 10 should normally be allowed, while requests beyond the configured limit should be rejected, depending on your exact rate-limit algorithm/window.

Postman

You can test the same endpoint repeatedly:

GET <http://localhost:8080/api/products>

Or use the **Postman Collection Runner** to send multiple requests automatically.

Step 9 — Verify Requests Above the Limit Are Blocked

Suppose:

Limit = 10 requests/minute

You send 15 requests.

Expected result:

Request Expected result

1	200 OK
2	200 OK
3	200 OK
...	...
10	200 OK
11	429
12	429
13	429
14	429
15	429

The important response is:

HTTP 429 Too Many Requests

What is HTTP 429

429 means:

The client has sent too many requests in a given period.

Example:

HTTP/1.1 429 Too Many Requests

You can return a JSON response such as:

```
{  
  "status": 429,  
  "error": "Too Many Requests",  
  "message": "Rate limit exceeded",  
  "retryAfter": 30
```

```
}
```

The exact JSON depends on your implementation.

What happens internally

Suppose the limit is:

10 requests / minute

The user sends requests.

Redis keeps track of the count:

User 101



Redis



rate:user:101 = 1

After more requests:

rate:user:101 = 5

Later:

rate:user:101 = 10

The next request reaches the Gateway:

Request



API Gateway



Redis



Count = 10



Limit = 10



Limit exceeded



HTTP 429

The request should **not be forwarded to the backend**.

Step 10 — Document the Rate-Limit Configuration

1. API being protected

Example:

Product API

GET /api/products

2. Rate-limit rule

Example:

100 requests / minute / user

Or for testing:

10 requests / minute / user

3. Redis configuration

Explain that Redis stores the rate-limit state/counters.

Example:

Redis

├— User ID

├— Request count

└— Expiration/time window

4. Gateway configuration

Explain that the API Gateway checks Redis before forwarding the request.

Client



API Gateway



Rate Limiter



Redis



Decision

5. 429 handling

Document that requests exceeding the configured limit receive:

429 Too Many Requests

6. Testing results

Document both successful and blocked requests.

Example:

Test	Result
Request within limit	200 OK
Request within limit	200 OK
Request reaches limit	200 OK
Request above limit	429
Repeated excessive requests	429